**Desarrollo de una plataforma web para la gestión centralizada de reservas, clientes y pagos del Hotel de Sandi**

# SandiStay - Sistema Integrado de Gestión Hotelera (PMS)

## Ingeniería de software II

**Marlon Duque Torrado**

# Contexto y Dominio del Problema

## Descripción de la Situación Actual

El sector de la hospitalidad, específicamente en hoteles de formato boutique o tamaño medio, enfrenta desafíos operativos críticos cuando su gestión depende de procesos manuales o herramientas ofimáticas desconectadas (hojas de cálculo, agendas físicas). En el caso de estudio que da origen a este proyecto, el hotel presentaba las siguientes problemáticas (Puntos de Dolor / Pain Points):

1. **Riesgo de Sobreventa (Overbooking):** La falta de un inventario centralizado y transaccional en tiempo real permitía que dos clientes (uno presencial y uno remoto) reservaran la misma habitación para las mismas fechas.
2. **Fricción en la Conversión de Ventas:** Los clientes debían llamar por teléfono o enviar mensajes para consultar disponibilidad y precios, lo que resultaba en pérdida de ventas fuera del horario de oficina.
3. **Descuadre Contable y Pagos Huérfanos:** La recepción de transferencias bancarias (Nequi/Daviplata) no estaba vinculada automáticamente a las reservas. Los clientes enviaban comprobantes que la recepción debía conciliar manualmente, generando demoras y errores humanos.
4. **Gestión de Áreas Sociales Ineficiente:** El hotel alquila espacios por horas (Salones de Eventos, Piscinas), pero el sistema manual no diferenciaba entre el modelo de cobro por noches (100% anticipado) y el modelo de cobro por horas (30% de depósito y saldo pendiente).

## Propuesta de Solución (SandiStay)

Para mitigar estas deficiencias, se propuso el diseño y desarrollo de **SandiStay**, un Sistema de Gestión de Propiedades (PMS - Property Management System) con arquitectura web Fullstack. La solución centraliza el inventario, automatiza la conciliación de pagos mediante integraciones de red, aplica reglas financieras estrictas para cancelaciones y reembolsos, y provee un portal público orientado a la conversión rápida mediante un flujo de Guest Checkout (reserva sin registro obligatorio).

# Análisis y Refactorización de Actores

Durante la fase inicial de elicitación de requisitos, los stakeholders (interesados) solicitaron la creación de roles específicos como "Cliente Regular" y "Cliente Premium", exigiendo que estos tuvieran un portal de inicio de sesión (Login) exclusivo.
Como parte del análisis de ingeniería, **este requisito fue refactorizado**. Se determinó que obligar a un cliente a crear una cuenta y recordar una contraseña genera una alta tasa de abandono en el embudo de ventas.

- **Solución Arquitectónica Aplicada:** Se unificaron todos los usuarios externos bajo el actor **"Cliente"**. La distinción de fidelidad (Premium, Regular, Corporativo) se modeló como un **Atributo de Datos** (idTipoCliente) en la base de datos. El sistema aplica los descuentos automáticamente basándose en este atributo, eliminando la necesidad de interfaces duplicadas y mejorando drásticamente la Experiencia de Usuario (UX).

Los actores definitivos del sistema son:

1. **Cliente (Huésped):** Actor externo. Interactúa con el portal público para consultar el catálogo, simular pagos y generar reservas. No posee credenciales de acceso al sistema.
2. **Secretaria (Staff):** Actor interno operativo. Gestiona el Check-in, Check-out, cancelaciones, cobro de saldos pendientes (Tesorería) y reservas presenciales (Walk-ins).

3. **Administrador (Admin):** Actor interno estratégico. Posee control total (CRUD) sobre el inventario (Habitaciones/Áreas), gestión de usuarios del sistema (Staff) y acceso a reportes financieros y de ocupación.

# Requisitos Funcionales del Sistema (FRQ)

Los requisitos funcionales definen el comportamiento exacto que el sistema debe proveer, divididos por módulos lógicos de negocio:

## Módulo 1: Catálogo y Disponibilidad

- **FRQ-01.1 - Búsqueda Dinámica:** El sistema debe permitir al Cliente buscar habitaciones (por rango de fechas y capacidad) y áreas sociales (por fecha y rango de horas).
- **FRQ-01.2 - Prevención de Overbooking:** El motor de base de datos debe garantizar matemáticamente que no se solapen dos reservas sobre el mismo activo en el mismo periodo de tiempo.
- **FRQ-01.3 - Gestión de Inventario (CRUD):** El Administrador debe poder crear, modificar y aplicar borrado lógico (Soft Delete) a habitaciones y áreas, así como subir y eliminar fotografías físicas asociadas a cada activo.
- **FRQ-01.4 - Control de Estado Operativo:** El Staff debe poder cambiar el estado de un activo (Ej. de "Disponible" a "Mantenimiento" o "Limpieza") siempre y cuando no existan reservas activas que entren en conflicto.

## Módulo 2: Motor de Reservas y Checkout

- **FRQ-02.1 - Guest Checkout (Wizard):** El sistema debe guiar al Cliente por un formulario de dos pasos: recolección de datos personales y procesamiento de pago, sin exigir creación de cuenta.
- **FRQ-02.2 - Deduplicación de Clientes:** Al recibir una reserva, el sistema debe buscar si el documento de identidad ya existe. Si existe, debe validar que el nombre y correo coincidan (Anti-suplantación) y asociar la reserva al historial del cliente existente.
- **FRQ-02.3 - Reglas de Cobro Polimórficas:** El sistema debe exigir el pago del 100% del valor total para reservas de Habitaciones, y un anticipo exacto del 30% para reservas de Áreas Sociales.
- **FRQ-02.4 - Reservas Presenciales (Walk-in):** El Staff debe contar con una interfaz dedicada para registrar reservas recibiendo el pago en efectivo, omitiendo la validación con pasarelas digitales.
    - Módulo 3: Tesorería y Lógica Financiera
- **FRQ-03.1 - Conciliación de Pagos:** El sistema debe comunicarse con un servicio externo (Mock de Pagos) enviando el número de comprobante ingresado por el cliente. Debe validar que el monto transferido coincida exactamente con el monto adeudado antes de confirmar la reserva.
- **FRQ-03.2 - Políticas de Cancelación Estrictas:** El sistema debe calcular penalidades automáticamente al cancelar una reserva:
    - Habitaciones: 35% de multa si se cancela con ≥ 5 días de anticipación; 100% si es menor.
    - Áreas Sociales: 15% del valor total si se cancela con ≥ 7 días; pérdida total del depósito (30%) si es menor.
- **FRQ-03.3 - Split Payment (División Contable):** Al cancelar, el sistema no debe borrar el pago original, sino anularlo lógicamente y generar dos nuevos registros: un ingreso por "Multa" y un pasivo por "Reembolso Pendiente".

- **FRQ-03.4 - Gestión de Cartera:** El Administrador debe tener un panel de Tesorería para registrar el pago de saldos pendientes (ej. el 70% restante de un Área Social) y procesar manualmente las devoluciones de dinero.

### Módulo 4: Reportes y Auditoría

- **FRQ-04.1 - Facturación PDF:** El sistema debe compilar y generar facturas comerciales en formato PDF, desglosando el subtotal, el IVA (19%) y el total pagado.
- **FRQ-04.2 - Reporte de Ocupación:** El sistema debe generar un reporte diario de ocupación e ingresos proyectados en un rango de fechas, incluyendo los días con 0% de ocupación.
- **FRQ-04.3 - Historial CRM:** El sistema debe proveer una vista analítica del cliente mostrando su total de reservas, tasa de cancelación y Lifetime Value (Total gastado en el hotel).

### Módulo 5: Seguridad y Control de Acceso

- **FRQ-05.1 - Gestión de Usuarios:** El Administrador debe poder registrar nuevas cuentas para el Staff y aplicar borrado lógico (desactivación) para revocar accesos.
- **FRQ-05.2 - Autogestión de Credenciales:** Todo usuario interno debe poder cambiar su propia contraseña validando su clave anterior.

## Requisitos No Funcionales del Sistema (NFR)

Los requisitos no funcionales establecen los atributos de calidad, seguridad y arquitectura que el sistema debe cumplir para ser considerado apto para producción.

### NFR-01: Seguridad (Security by Design)

- **Autenticación Stateless:** El sistema debe utilizar JSON Web Tokens (JWT) firmados con HMAC-SHA256 para la gestión de sesiones del personal.
- **Protección contra XSS y CSRF:** Los tokens JWT deben almacenarse exclusivamente en Cookies HttpOnly y SameSite=Strict. El Frontend no debe tener acceso al token mediante JavaScript.
- **Control de Acceso Basado en Roles (RBAC):** Las rutas del Frontend y los endpoints del Backend deben estar protegidos jerárquicamente. Un usuario STAFF no debe poder renderizar ni consumir recursos de nivel ADMIN.
- **Cifrado de Datos:** Las contraseñas de los usuarios deben ser hasheadas utilizando el algoritmo **BCrypt** con un Work Factor mínimo de 10.
- **Ocultación de Errores (Information Disclosure):** El sistema no debe exponer trazas de la base de datos (Stack Traces o SQL) al cliente bajo ninguna circunstancia de error (Ej. violaciones de constraints).

### NFR-02: Arquitectura y Rendimiento (Performance)

- **Fat Database / Thin Controller:** La lógica de cálculo masivo (totales, ocupación, cruce de fechas) debe ejecutarse en el motor de base de datos mediante Vistas (VIEW) y Procedimientos Almacenados (PL/pgSQL), minimizando el procesamiento en la memoria de la aplicación (Java).
- **Server-Side Rendering (SSR):** El catálogo público debe ser renderizado en el servidor (Node.js) antes de ser enviado al cliente para garantizar tiempos de carga óptimos (LCP) y un correcto posicionamiento en buscadores (SEO).

- **Gestión de Archivos (Multipart):** Las imágenes del catálogo deben almacenarse físicamente en el disco del servidor (Host) y no en la base de datos, con un límite de subida configurado a 20MB en todas las capas de red.

### NFR-03: Despliegue e Infraestructura (DevOps)

- **Containerización:** Todos los componentes del sistema (Base de Datos, Backend, Frontend, Mock de Pagos y API Gateway) deben estar empaquetados en contenedores Docker utilizando Multi-stage builds basados en Alpine Linux para reducir el tamaño de las imágenes.
- **Aislamiento de Red (Zero Trust):** La base de datos y los servicios internos no deben exponer puertos al exterior. Todo el tráfico entrante debe ser gestionado y filtrado por un proxy inverso (Nginx) en el puerto 80/443.
- **Portabilidad (12-Factor App):** La configuración del sistema (URLs, credenciales, secretos) debe inyectarse mediante variables de entorno (.env), garantizando que el código fuente sea inmutable entre los entornos de Desarrollo, Pruebas y Producción.

# Metodología de Desarrollo

Para la ejecución del proyecto SandiStay, se seleccionó el **Modelo en V (V-Model) Extendido**, complementado con prácticas de ejecución ágil (Kanban). Esta decisión metodológica se fundamenta en la naturaleza del proyecto: al ser desarrollado por un único ingeniero asumiendo un rol Fullstack Integral, se requería un marco de trabajo que priorizara la validación temprana de la arquitectura y la trazabilidad estricta entre los requisitos y las pruebas, evitando el retrabajo asociado a metodologías puramente iterativas.
El ciclo de vida se estructuró en las siguientes fases:

1. **Fase de Definición (Brazo Izquierdo de la V):** Levantamiento de requisitos, análisis de viabilidad y diseño arquitectónico detallado (Base de Datos, API y Seguridad).
2. **Vértice de Ejecución:** Planificación de la Estructura de Desglose de Trabajo (EDT) y estimación de costos.
3. **Fase de Construcción y Validación (Brazo Derecho de la V):** Desarrollo de código (Backend y Frontend), pruebas de integración continua (Smoke Tests) y despliegue en contenedores.

# Diseño del Sistema y Arquitectura

El diseño del sistema se abordó desde una perspectiva Cloud-Native, garantizando que la aplicación sea escalable, segura y fácil de mantener.

### Arquitectura de Software (Patrones de Diseño)

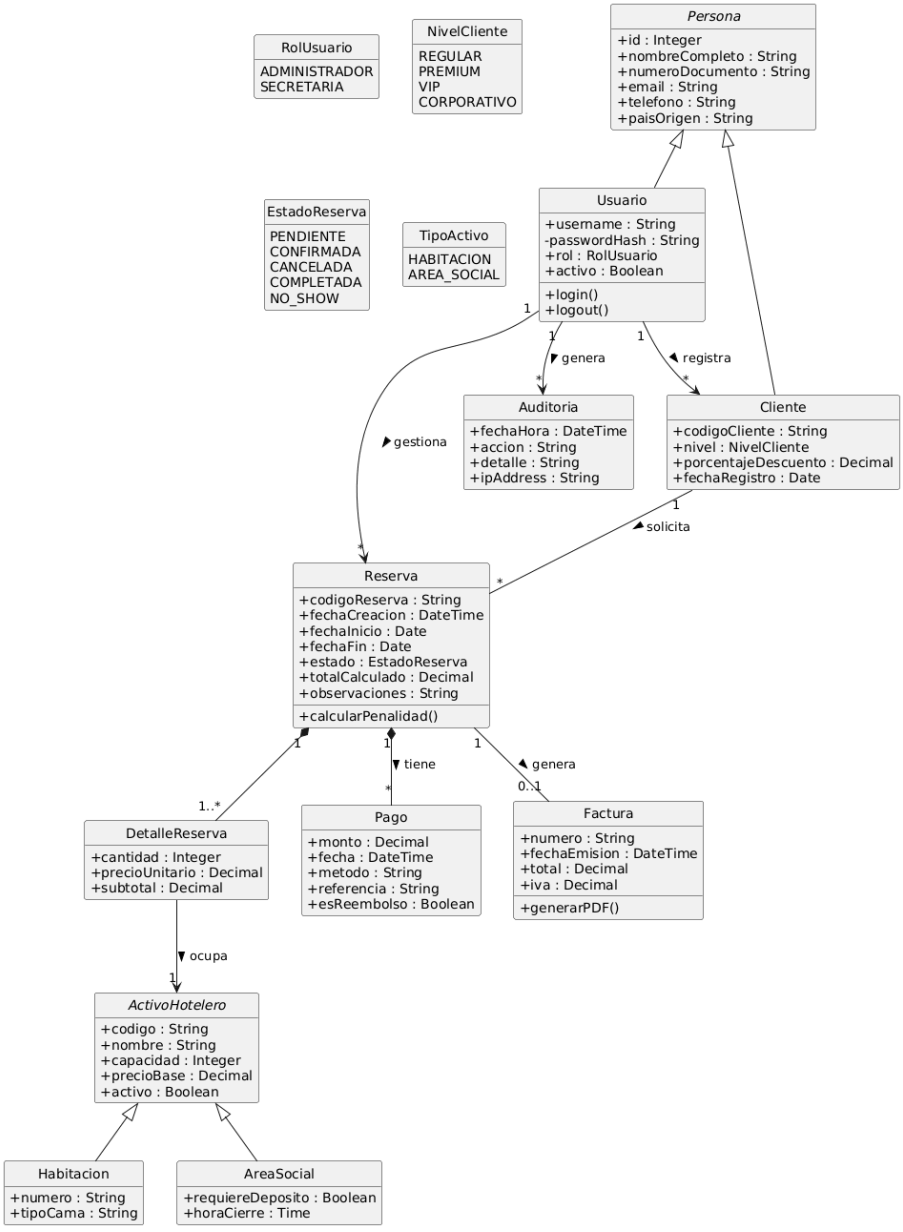
El sistema se construyó bajo el patrón de **Monolito Modular (Modular Monolith)**. A diferencia de una arquitectura MVC tradicional agrupada por capas técnicas (todos los controladores juntos), el código se agrupó por **Dominios de Negocio (Bounded Contexts)**.

- **Backend (Spring Boot):** Se estructuró en módulos independientes (auth, bookings, rooms, finance), compartiendo únicamente un núcleo transversal (shared kernel) para el manejo de excepciones y utilidades.
- **Frontend (SvelteKit):** Se implementó el patrón de **Route Groups** ((app) para el portal público y (admin) para el backoffice). Esto permitió aplicar Layout Inheritance (herencia de plantillas) y aislar la seguridad visual sin afectar la estructura de las URLs.

## Diseño de Base de Datos ("Fat Database")

Se rechazó el enfoque de usar la base de datos como un simple almacén de datos pasivo. En su lugar, se diseñó una arquitectura donde PostgreSQL actúa como el motor de reglas de negocio pesadas.

- Se implementaron **Constraints de Exclusión (EXCLUDE USING gist)** para prevenir matemáticamente el Overbooking a nivel de motor.
- Se desarrollaron **Vistas SQL (VIEW)** pre-calculadas para los reportes financieros y **Procedimientos Almacenados (PL/pgSQL)** para garantizar la atomicidad en operaciones complejas como el Check-out y la facturación.

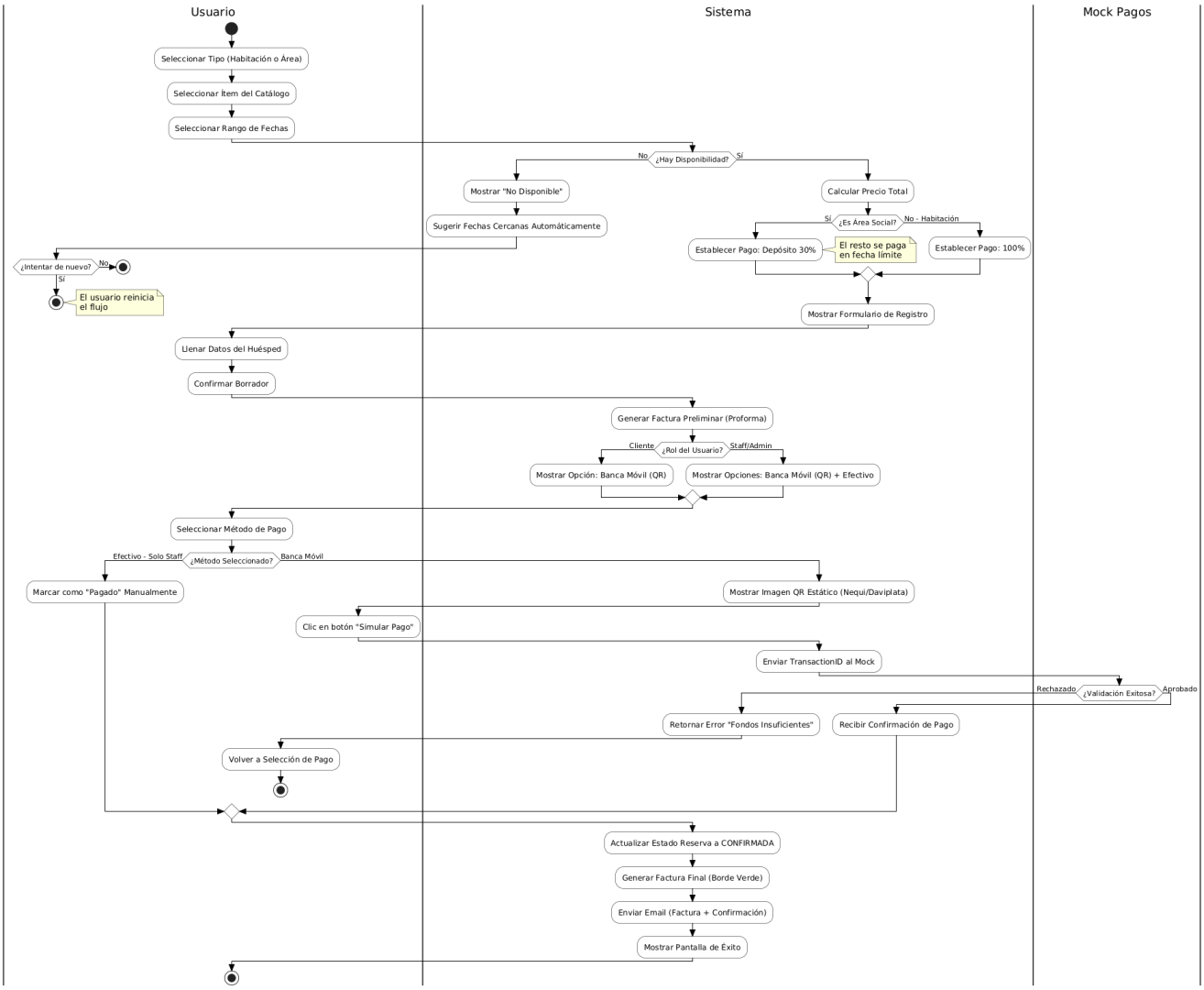


# Diseño de Interacciones y Lógica de Negocio

Para modelar el comportamiento dinámico del sistema, se diseñaron diagramas de actividad que orquestan la interacción entre los actores, el sistema central y los servicios externos.

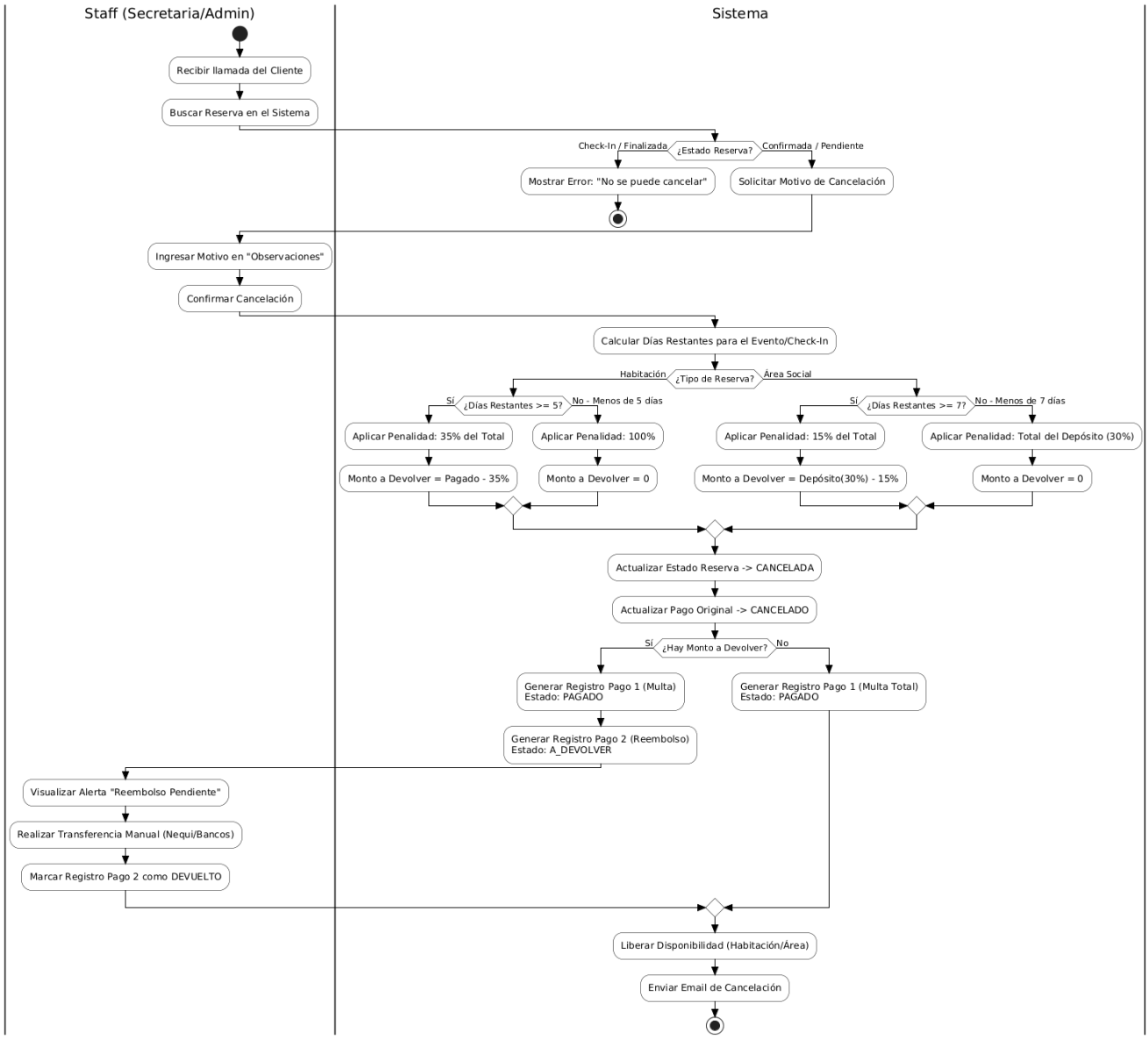
**A. Flujo de Reserva y Conciliación Bancaria**

El proceso de reserva se diseñó para soportar pagos asíncronos y validación anti-fraude. El sistema genera una intención de pago, presenta un código QR estático y delega la validación a un microservicio externo (Mock de Pagos) antes de confirmar la transacción.




**B. Flujo de Cancelación y Split Payment**

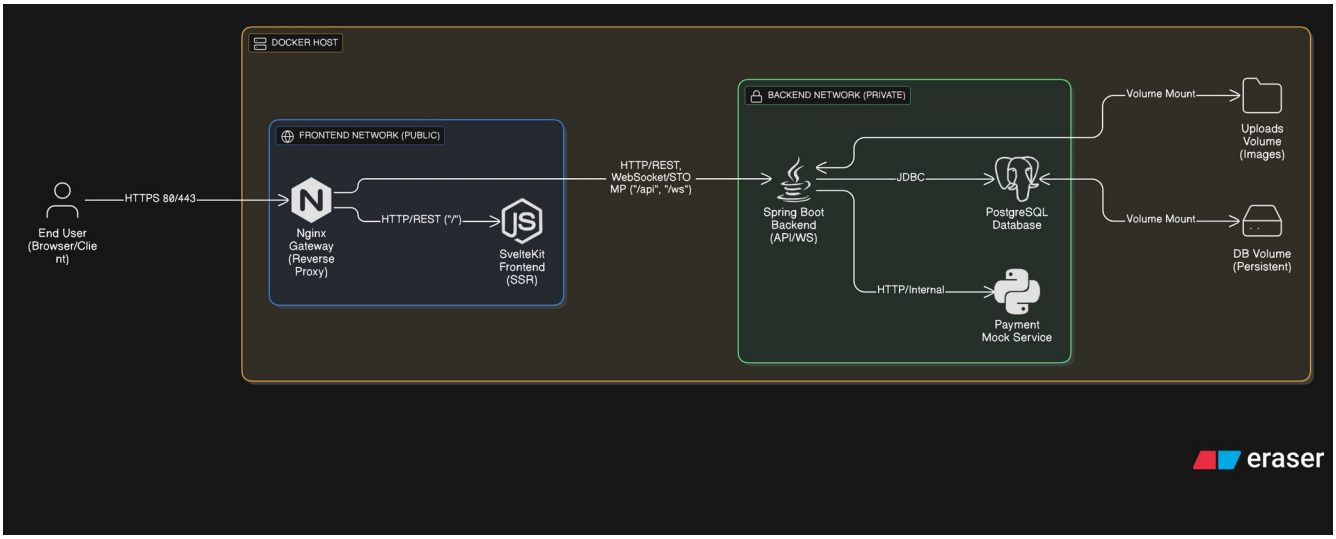
La lógica de cancelación implementa reglas de penalidad basadas en ventanas de tiempo (5 días para habitaciones, 7 días para áreas). Contablemente, el sistema aplica un patrón de Split Payment: anula lógicamente el pago original y genera dos nuevos registros (Multa y Reembolso Pendiente) para mantener la integridad del libro mayor.




## Diseño de Infraestructura y Despliegue

La topología de producción se diseñó bajo el principio de **Zero Trust Network** (Red de Confianza Cero) utilizando Docker Compose.

- Se definieron 5 contenedores aislados: Base de Datos, Mock de Pagos, Backend (Java), Frontend (Node.js) y API Gateway (Nginx).
- El único puerto expuesto a internet es el puerto 80 del Gateway. La comunicación entre el Frontend (SSR) y el Backend ocurre exclusivamente a través del DNS interno de Docker, mitigando vectores de ataque externos.




# Planeación y Gestión del Proyecto

La ejecución del proyecto se planificó detallando el esfuerzo técnico requerido y modelando un presupuesto realista ajustado al mercado colombiano.

## Estructura de Desglose de Trabajo (EDT / WBS)

El trabajo de construcción se dividió en 6 paquetes principales, secuenciados lógicamente para evitar bloqueos por dependencias:

1. Configuración del Entorno (Git, Docker, Init).
2. Implementación de Base de Datos (DDL, PL/pgSQL, Seed Data).
3. Desarrollo Backend (API REST, Seguridad, Lógica Transaccional).
4. Desarrollo Frontend (UI Kit, Portal Público, Dashboard Admin).
5. Pruebas y Aseguramiento de Calidad (QA).
6. Despliegue (Nginx, Multi-stage Dockerfiles).

## Cronograma de Desarrollo

El cronograma se diseñó bajo un modelo de **Cascada Estricta (Finish-to-Start)**, asumiendo una capacidad operativa de 2 horas diarias (perfil freelance) y excluyendo fines de semana y festivos nacionales. Se aplicó la regla de redondeo hacia arriba para generar holguras (buffers) y evitar el desgaste cognitivo por Context Switching.


Cronograma de Desarrollo - SandiStay (V-Model)
Estructura de Desglose de Trabajo (EDT)
0.0 Fases Previas (Requisitos, Análisis y Diseño)
0.1 Ejecución de Fases 1 a 4
1.0 Configuración del Entorno
1.1 Configuración Repositorio
1.2 Docker Compose Local
1.3 Init Backend (Spring)
1.4 Init Frontend (Svelte)
2.0 Implementación de Base de Datos
2.1 DDL Tablas y Relaciones
2.2 Constraints Avanzados
2.3 Programación PL/pgSQL
2.4 Datos Semilla (Seed)
3.0 Desarrollo Backend (Spring Boot - Logic Layer)
3.1 Módulo Transversal (Shared Kernel)
3.1.1 Configuración de Seguridad
3.1.2 Manejo Global Excepciones
3.2 Módulo de Autenticación (Auth)
3.2.1 Endpoint Login
3.2.2 Endpoint Perfil (/me)
3.3 Módulo de Catálogo (Rooms/Areas)
3.3.1 Búsqueda y Filtros
3.4 Módulo de Reservas (Core)
3.4.1 Lógica de Creación
3.4.2 Lógica de Cancelación
3.4.3 Integración Mock Pagos
3.5 Módulo de Reportes
3.5.1 Generación de PDFs
4.0 Desarrollo Frontend
4.1 Estructura y Componentes Base
4.1.1 Layouts Base
4.1.2 Componentes UI
4.2 Portal Público (Cliente)
4.2.1 Inicio y Búsqueda
4.2.2 Detalle de Habitación
4.2.3 Flujo de Reserva
4.3 Portal Administrativo (Staff)
4.3.1 Login y Guards
4.3.2 Dashboard Reservas
4.3.3 Gestión Check-in/out
4.3.4 Vista de Reportes
5.0 Pruebas (QA)
5.1 Unit Testing Backend
5.2 Integration Testing
5.3 E2E Frontend
5.4 Security Testing
6.0 Despliegue
6.1 Configuración VPS
6.2 Dominio y SSL
6.3 Despliegue Final
Hito 0: Planificación y Diseño Aprobados (Baseline)
Hito 1: Base de Datos y API Funcional
Hito 2: Interfaz de Usuario Completada
Hito 3: Go-Live (Paso a Producción)
17-Feb-2026
24-Feb-2026
03-Mar-2026
10-Mar-2026
17-Mar-2026
24-Mar-2026
31-Mar-2026
07-Apr-2026
14-Apr-2026
21-Apr-2026
28-Apr-2026
05-May-2026
12-May-2026
19-May-2026
26-May-2026
02-Jun-2026
09-Jun-2026
16-Jun-2026
23-Jun-2026
30-Jun-2026
07-Jul-2026
14-Jul-2026
21-Jul-2026
28-Jul-2026
04-Aug-2026
11-Aug-2026
18-Aug-2026
25-Aug-2026
Línea de Tiempo

## Estimación de Costos y Presupuesto (TCO)

El presupuesto se calculó diferenciando los costos de infraestructura (OPEX) y los honorarios de ingeniería de software. Se aplicaron tarifas de mercado para perfiles Senior (30.000COP/h para Arquitectura y Gestion) y Junior/Mid(14.000 COP/h para Construcción).
Para garantizar la viabilidad legal y comercial del proyecto, se incluyó el cálculo exacto de la Seguridad Social colombiana (PILA sobre 1 SMMLV) y un margen de utilidad operativa del 30%.

**Resumen Presupuestal del Proyecto (Año 1)**

| Categoría | Descripción | Costo Total (COP) | % del Total |
|---|---|---|---|
| **Ingeniería de Software** | Honorarios (290h) + Seguridad Social (5 meses) + Utilidad (30%). | $ 10.764.962 | 76,2% |
| **Infraestructura Cloud** | Operación del servidor VPS por 12 meses (+20% Margen). | $ 2.067.840 | 14,7% |
| **Imprevistos** | Margen de riesgo (10% sobre Infra + Software). | $ 1.283.280 | 9,1% |
| **GRAN TOTAL** | **Costo Total de Propiedad (TCO)** | **$ 14.116.082** | **100%** |

# Desarrollo del Sistema (Construcción)

La fase de construcción se ejecutó siguiendo un enfoque Bottom-Up (de abajo hacia arriba) y Contract-First (definiendo primero las interfaces de comunicación), garantizando que cada capa del sistema fuera funcional y testeable antes de integrarse con la siguiente. El stack tecnológico utilizado incluyó **Java 25, Spring Boot 4.0.5, Svelte 5, Tailwind CSS v4 y PostgreSQL 17**.

## Capa de Datos (PostgreSQL)

Se materializó el diseño de "Fat Database" mediante la ejecución de scripts DDL y DML estructurados.

- **Lógica Interna:** Se programaron funciones (fn_buscar_habitaciones_disponibles, fn_reporte_ocupacion) y procedimientos almacenados (sp_cancelar_reserva, sp_completar_reserva) en **PL/pgSQL**. Esto permitió que cálculos financieros complejos (como la extracción del IVA o el cálculo de multas) se ejecutaran directamente en el motor de base de datos, reduciendo la latencia de red y el consumo de memoria en el servidor de aplicaciones.
- **Datos Semilla (Seed Data):** Se automatizó la carga de catálogos base, roles de seguridad y el usuario administrador inicial mediante el mapeo del directorio /docker-entrypoint-initdb.d/ en el contenedor de base de datos.

## Capa de Lógica de Negocio (Backend - Spring Boot)

El Backend se construyó como una API RESTful robusta, actuando como orquestador de seguridad y transacciones.

- **Seguridad y Autenticación:** Se implementó un filtro personalizado (JwtAuthenticationFilter) utilizando la librería jjwt:0.12.5. Se configuró Spring Security para proteger los endpoints mediante Control de Acceso Basado en Roles (RBAC) usando anotaciones @PreAuthorize. Las contraseñas fueron aseguradas mediante el algoritmo **BCrypt**.
- **Manejo de Excepciones:** Se desarrolló un GlobalExceptionHandler (@RestControllerAdvice) para interceptar errores internos y excepciones de base de datos

(DataIntegrityViolationException), transformándolos en respuestas JSON estandarizadas (RFC 7807) y evitando la fuga de información sensible (Information Disclosure).
- **Generación de Documentos:** Se integró el motor **JasperReports** para la compilación en memoria de plantillas XML (.jrxml), permitiendo la generación dinámica de facturas comerciales y reportes gerenciales en formato PDF.

### Capa de Presentación (Frontend - SvelteKit)

La interfaz de usuario se desarrolló priorizando el rendimiento (Performance) y la Experiencia de Usuario (UX).
- **Sistema de Diseño:** Se aplicaron los principios de Atomic Design para crear una biblioteca de componentes UI reutilizables (Button, Input, Card) estilizados con Tailwind CSS v4.
- **Server-Side Rendering (SSR) y Server Actions:** Para proteger el Token JWT contra ataques XSS, la autenticación se manejó exclusivamente en el servidor de Node.js, almacenando el token en una **Cookie HttpOnly**. Las mutaciones de datos (Check-in, Cancelaciones) se implementaron mediante Server Actions, garantizando que la aplicación funcione de manera resiliente.
- **Route Guards:** Se implementó el archivo hooks.server.ts como un guardián centralizado que intercepta todas las peticiones de navegación, redirigiendo al usuario a la pantalla de login si intenta acceder a rutas protegidas (/dashboard, /settings) sin autorización.

### Microservicio de Conciliación Bancaria (Mock Python)

Para simular un entorno financiero real sin depender de pasarelas de pago en producción, se desarrolló un microservicio independiente utilizando **Python y FastAPI**.
- Este servicio actúa como el "Libro Mayor" del banco. Implementa lógica determinista basada en el número de comprobante ingresado por el usuario (ej. EXACTO-, FRAUDE-, DEMORA-) para simular escenarios de fondos insuficientes, latencia de red (Timeouts) y pagos exitosos, permitiendo probar la resiliencia del Backend en Java.

# Aseguramiento de Calidad y Pruebas (QA & Testing)

En lugar de depender exclusivamente de pruebas unitarias aisladas, la estrategia de QA se centró en **Pruebas de Integración (Smoke Tests)** y **Pruebas de Aceptación de Usuario (UAT)**. Este enfoque permitió validar el comportamiento del sistema en su conjunto, detectando fallos arquitectónicos que solo ocurren cuando múltiples contenedores interactúan entre sí.
A continuación, se documentan los incidentes críticos (Bugs) detectados durante las pruebas y su respectiva resolución arquitectónica (Troubleshooting):

### Incidentes de Lógica de Negocio y Base de Datos

1. **Fallo de Integridad en Cancelaciones (Deuda Fantasma):**
   - Incidente: Al cancelar una reserva, el sistema calculaba la multa pero mantenía el saldo pendiente original, enviando al cliente a la bandeja de "Cuentas por Cobrar" en Tesorería.
   - Resolución: Se refactorizó el método cancelBooking en Java para aplicar un patrón de Split Payment estricto. Se anularon lógicamente todos los pagos previos y se generaron nuevos registros compensatorios (Multa y Reembolso), ajustando la vista SQL para excluir reservas canceladas de la cartera activa.
2. **Ambigüedad SQL en Reportes (Error 500):**
   - Incidente: La función fn_reporte_ocupacion fallaba con el error column reference "ocupadas" is ambiguous al utilizar Common Table Expressions (CTEs).

- Resolución: Se aplicó un estándar de nomenclatura estricto en PL/pgSQL, agregando el prefijo out_ a las variables del RETURNS TABLE para evitar colisiones de nombres con las columnas internas del SELECT.

3. **Validación de Overbooking (Error 409 Conflict):**
   - Incidente: Al intentar reservar una habitación en fechas ya ocupadas, el sistema arrojaba una traza SQL completa al Frontend.
   - Resolución: Se confirmó que el constraint EXCLUDE USING gist de PostgreSQL funcionaba perfectamente. Se ajustó el GlobalExceptionHandler en Java para atrapar esta excepción específica y devolver un mensaje amigable al usuario, protegiendo la topología de la base de datos.

### Incidentes de Integración y Generación de Archivos

1. **Fallo de Compilación en JasperReports (Invalid UUID):**
   - Incidente: El motor de reportes rechazaba la plantilla .jrxml arrojando un error fatal durante la generación del PDF.
   - Resolución: Se identificó que la versión 6.21.3 de JasperReports exige validación estricta de identificadores. Se reemplazaron los valores de relleno (dummy) por UUIDs válidos (formato 8-4-4-4-12) en el código XML.
2. **PDF Generado en Blanco (Transaction Isolation):**
   - Incidente: El PDF de la factura se descargaba correctamente pero sin datos, a pesar de que la base de datos confirmaba la inserción de los registros.
   - Resolución: Se diagnosticó un problema de aislamiento de transacciones (READ COMMITTED). JasperReports abría una nueva conexión al pool que no podía leer los datos no confirmados por Spring. Se inyectó DataSourceUtils.getConnection() para obligar al motor de reportes a compartir el mismo hilo transaccional activo de Spring Boot.

### Incidentes de Infraestructura y Redes Docker (DevOps)

1. **Timeouts por Hairpin NAT en Server-Side Rendering:**
   - Incidente: SvelteKit fallaba al intentar consumir la API de Spring Boot durante el renderizado en el servidor (SSR) dentro del entorno Docker de producción.
   - Resolución: Se implementó el hook handleFetch en SvelteKit para interceptar las peticiones SSR y enrutarlas directamente a través del DNS interno de Docker (http://backend:8082), evitando que la petición saliera a internet y reingresara por el API Gateway (Nginx).
2. **Bloqueo CORS por Origen Nulo (Origin: null):**
   - Incidente: Spring Security bloqueaba las peticiones internas de Node.js devolviendo 403 Invalid CORS Request.
   - Resolución: Se forzó la inyección explícita de la cabecera Origin en el handleFetch de SvelteKit, simulando el contexto de un navegador para satisfacer los filtros de seguridad de Java.

## Fases Pendientes y Trabajo Futuro (Backlog)

Si bien el núcleo transaccional y las interfaces de usuario han sido construidas y estabilizadas, el ciclo de vida del **Modelo en V (V-Model)** requiere la ejecución de las etapas finales del brazo derecho de la "V" para garantizar la entrega exitosa del producto al cliente final. Las siguientes fases y artefactos se encuentran programados para su ejecución:

## Fase 5: Pruebas Formales y Aseguramiento de Calidad (QA)

Aunque durante la fase de desarrollo se ejecutaron pruebas de integración y Smoke Tests exhaustivos que garantizaron la estabilidad de la arquitectura, es imperativo ejecutar una fase de QA formal y automatizada antes del paso a producción:

- **Pruebas Unitarias y de Integración (Backend):** Implementación de Testcontainers y JUnit 5 para validar la lógica financiera y los Constraints de la base de datos en entornos efímeros, asegurando que futuras actualizaciones no rompan el código existente (Pruebas de Regresión).
- **Pruebas End-to-End (Frontend):** Automatización de flujos críticos (ej. Guest Checkout y Cancelaciones) utilizando **Playwright**, simulando el comportamiento real de un usuario en diferentes navegadores y dispositivos móviles.
- **Pruebas de Carga y Estrés:** Simulación de concurrencia masiva para validar que el motor de PostgreSQL maneje correctamente los bloqueos transaccionales y evite el Overbooking bajo alta demanda.

## Fase 6: Despliegue en Producción (Go-Live)

El sistema ya se encuentra empaquetado en contenedores Docker (Multi-stage builds) y orquestado mediante docker-compose.prod.yml. El trabajo restante consiste en el aprovisionamiento de la infraestructura física:

- **Aprovisionamiento Cloud:** Configuración del servidor VPS en DigitalOcean (4GB RAM, 2 vCPU) y aplicación de políticas de Hardening en el sistema operativo Linux (Firewall UFW, deshabilitación de root login).
- **Gestión de Redes y Seguridad:** Configuración del dominio oficial del hotel, enrutamiento DNS y emisión de certificados SSL/TLS automatizados mediante Let's Encrypt integrados en el API Gateway (Nginx).
- **Pipelines CI/CD:** Implementación de flujos de Integración y Despliegue Continuo (ej. GitHub Actions) para automatizar la subida de nuevas versiones al servidor sin tiempo de inactividad (Zero Downtime Deployment).

## Documentación Final y Capacitación

La adopción del sistema por parte del personal del hotel es tan crítica como el código mismo. Queda pendiente la entrega de:

- **Manual de Usuario y Operaciones:** Un documento visual y detallado dirigido a la Secretaria y al Administrador. Explicará paso a paso cómo gestionar el Check-in/Check-out, cómo procesar reembolsos en el módulo de Tesorería y cómo interpretar los reportes de ocupación.
- **Manual Técnico de Despliegue:** Documentación dirigida a futuros desarrolladores o administradores de sistemas (SysAdmins) con las instrucciones exactas para levantar, respaldar (Backups) y restaurar la base de datos y los contenedores.

## Evolución del Producto (Roadmap Futuro)

Para versiones posteriores (Release 2.0), se contempla reemplazar el microservicio Mock de Python por integraciones directas con las APIs de producción de pasarelas de pago reales (Wompi, ePayco o Nequi API), así como la implementación de un servidor SMTP (ej. SendGrid) para el envío automatizado de correos electrónicos con las facturas PDF adjuntas.

# Conclusión General

El desarrollo del sistema **SandiStay** demuestra de manera contundente la viabilidad de aplicar arquitecturas de software de nivel empresarial (Enterprise-Grade) en el sector de la hospitalidad boutique.

A lo largo de este proyecto, se logró transformar un conjunto de requisitos de negocio complejos y propensos a errores humanos —como la sobreventa de habitaciones, la conciliación manual de pagos y la pérdida de trazabilidad financiera— en una solución digital automatizada, segura y altamente cohesiva.

La decisión de adoptar el patrón arquitectónico **"Fat Database / Thin Controller"** probó ser un éxito rotundo. Al delegar la responsabilidad de la integridad referencial, los cálculos financieros y la prevención de Overbooking directamente al motor de **PostgreSQL** (mediante Triggers, Vistas y Procedimientos Almacenados), se construyó un Backend en **Spring Boot** extremadamente ligero, rápido y libre de lógica redundante.

Por su parte, la implementación del Frontend con **SvelteKit** y **Tailwind CSS** garantizó una experiencia de usuario (UX) fluida y moderna. La aplicación estricta de principios de ciberseguridad, como el uso de Server Actions para el manejo de tokens JWT en cookies HttpOnly y la implementación de Route Guards, asegura que el sistema esté protegido contra las vulnerabilidades más críticas del OWASP Top 10.

Finalmente, la orquestación de toda la infraestructura mediante **Docker** asegura que el proyecto no sufra del clásico problema "funciona en mi máquina". SandiStay es hoy un sistema Cloud-Native, modular y resiliente, preparado para superar las fases de pruebas formales y ser desplegado en producción para transformar la operación diaria del hotel.